Credit Card Approval Prediction using Machine Learning

Project Description:

Credit Card Approval Prediction is a Machine Learning-based web application developed to assist financial institutions in evaluating credit card applications efficiently. The system predicts whether a customer's credit card application is likely to be approved or rejected based on applicant information such as gender, income, education, employment status, family details, housing type, and credit history.

The application uses a **Random Forest Classifier** trained on publicly available credit card application and credit history datasets. Data preprocessing techniques such as missing value handling, label encoding, feature scaling, and train-test splitting were applied before training the model. The trained model is integrated with a **Flask** web application that provides a simple and userfriendly interface for users to enter applicant details and receive instant prediction results.

The project is developed using **Python, Flask, HTML, CSS, Pandas, NumPy, Scikit-learn, and Joblib**. Version control is maintained using Git and GitHub to enable collaborative development and proper project management.

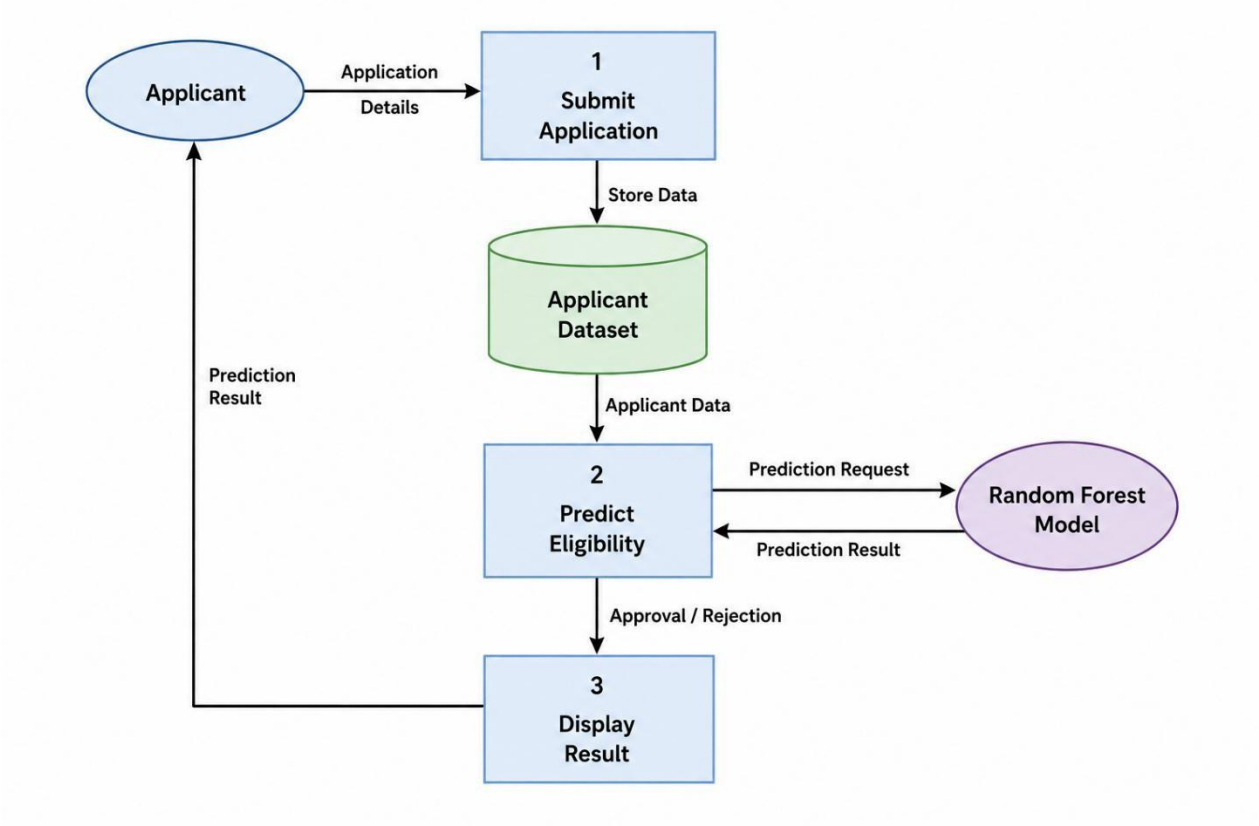
Scenarios

Scenario 1: A bank employee receives a credit card application from a salaried applicant with a stable income and good employment history. After entering the applicant's details into the system, the model predicts **Approved**, helping the bank process the application quickly.

Scenario 2: An applicant with very low income, unstable employment history, and poor financial information submits a credit card application. After entering the details, the system predicts **Rejected**, allowing the bank to identify high-risk applications efficiently.

Scenario 3: A financial institution processes multiple customer applications every day. Instead of manually reviewing every application, employees use the Credit Card Approval Prediction system to obtain instant approval predictions, reducing processing time while maintaining consistency in preliminary decision-making.

Technical Architecture:



Description: The Credit Card Approval Prediction system follows a Machine Learning-based architecture for predicting whether a customer's credit card application is approved or rejected. The user enters applicant details through a Flask-based web interface developed using HTML and CSS. The application validates the user input and performs the same preprocessing steps used during model training, including feature scaling using StandardScaler.

The processed data is passed to the trained Random Forest Classifier, which generates the prediction result. The prediction is then displayed on the result page as either **Approved** or **Rejected** along with the model's confidence score. The trained model and scaler are stored using the Joblib library, enabling efficient loading without retraining. GitHub is used for version control and collaborative project management.

Pre-requisites:

- Python 3.10 or later
- Flask Framework
- Pandas
- NumPy
- Scikit-learn
- Joblib
- Git & GitHub
- Visual Studio Code

Project Workflow:

Milestone 1: Data Collection and Preprocessing

- **Activity 1.1:** Collected the application_record.csv and credit_record.csv datasets.
- **Activity 1.2:** Merged both datasets and cleaned missing values.
- **Activity 1.3:** Performed label encoding for categorical features.
- **Activity 1.4:** Applied feature scaling using StandardScaler and balanced the training data using SMOTE.

Milestone 2: Machine Learning Model Development

- **Activity 2.1:** Split the dataset into training and testing sets.
- **Activity 2.2:** Trained the Random Forest Classifier using the processed training dataset.
- **Activity 2.3:** Evaluated the model using Accuracy, Precision, Recall, F1-Score, ROC-AUC Score, Confusion Matrix, and Classification Report.

Milestone 3: Flask Application Development

- **Activity 3.1:** Developed the Flask backend and integrated the trained machine learning model.
- **Activity 3.2:** Loaded the trained model and scaler using Joblib to perform real-time predictions.

Milestone 4: Frontend Development

- **Activity 4.1:** Designed responsive web pages using HTML and CSS.
- **Activity 4.2:** Developed the applicant input form and result page to display approval or rejection predictions with confidence score.

Milestone 5: Testing and Integration

- **Activity 5.1:** Tested the complete workflow using different applicant records.
- **Activity 5.2:** Verified model predictions, input validation, and application functionality under various test scenarios.

Milestone 6: Conclusion

The Credit Card Approval Prediction system was successfully developed using a Random Forest Classifier and integrated with a Flask web application. The system accurately predicts credit card approval status based on applicant details and provides a simple, user-friendly interface for real-time prediction. The project was completed successfully with proper testing and documentation.

Milestone 1: Model Selection and Architecture

In this milestone, the appropriate machine learning algorithm for predicting credit card approval was selected. Different classification algorithms were studied and compared before choosing Random Forest Classifier due to its high accuracy, robustness, and ability to handle mixed numerical and categorical features. The overall application architecture was also designed, integrating the trained model with a Flask web application for real-time prediction.

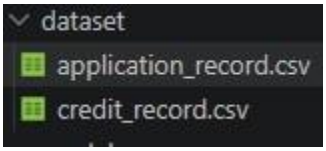
Activity 1.1: Dataset Collection and Understanding

Objective:

Collect and understand the dataset required for predicting credit card approval.

Steps Performed

- Downloaded the **Application Record** and **Credit Record** datasets.
- Studied all applicant attributes including income, education, occupation, family status, and employment details.
- Understood the target variable by combining applicant information with credit history.
- Verified data quality before preprocessing.



Activity 1.2: Data Preprocessing

Steps Performed

- Removed unnecessary columns.
- Handled missing values.
- Applied Label Encoding to categorical features.
- Standardized numerical features using StandardScaler.
- Balanced the dataset using SMOTE to reduce class imbalance.
- Split the dataset into Training and Testing sets.

```
# Fill missing values in Occupation Type

df["OCCUPATION_TYPE"] = df["OCCUPATION_TYPE"].fillna("Unknown")

# Check remaining missing values

df.isnull().sum()
```

[16]

```
ID                    0
CODE_GENDER           0
FLAG_OWN_CAR          0
FLAG_OWN_REALTY       0
CNT_CHILDREN          0
AMT_INCOME_TOTAL      0
NAME_INCOME_TYPE      0
NAME_EDUCATION_TYPE   0
NAME_FAMILY_STATUS    0
NAME_HOUSING_TYPE     0
DAYS_BIRTH            0
DAYS_EMPLOYED         0
FLAG_MOBIL            0
FLAG_WORK_PHONE       0
FLAG_PHONE            0
FLAG_EMAIL            0
OCCUPATION_TYPE       0
CNT_FAM_MEMBERS       0
TARGET                0
dtype: int64
```

Activity 1.3: Model Selection and Training

Steps Performed

• Compared different classification algorithms.

• Selected **Random Forest Classifier** because it produced stable performance.

• Trained the model using the processed training dataset.

• Evaluated the model using Accuracy, Precision, Recall, F1-Score, ROC-AUC Score, Confusion Matrix, and Classification Report.

• Saved the trained model using Joblib.

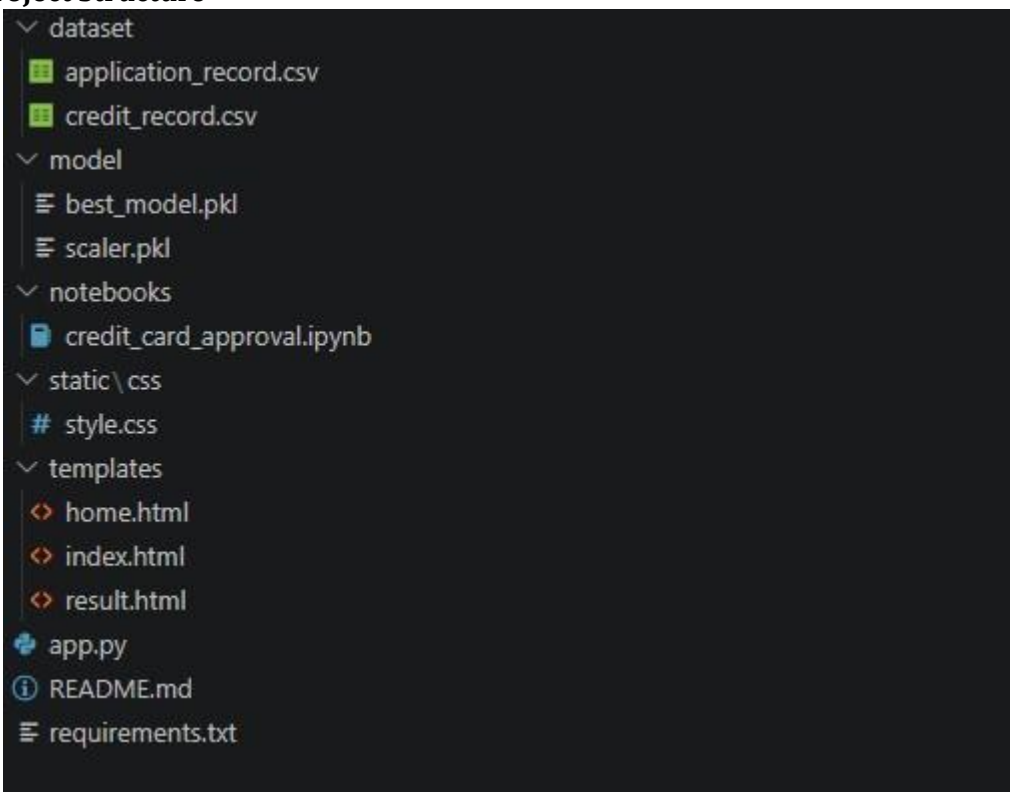
	Metric	Score
0	Accuracy	0.7505
1	Precision	0.9181
2	Recall	0.7875
3	F1-Score	0.8478
4	ROC-AUC Score	0.6812

Activity 1.4: Development Environment Setup

Development Environment

- Python 3.12
- Visual Studio Code
- Jupyter Notebook
- Flask Framework
- Scikit-learn
- Pandas
- NumPy
- Joblib

Project Structure



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

The core functionality of the application was developed to predict whether a credit card application should be approved or rejected based on applicant information.

The implemented features include:

- **Applicant Information Form:** Collects applicant details such as gender, income, employment type, education, family status, housing type, and family members.
- **Eligibility Validation:** Performs basic validation rules before prediction, such as minimum age, valid income, and employment checks.
- **Machine Learning Prediction:** Uses the trained Random Forest model to classify the application as Approved or Rejected.
- **Confidence Score:** Displays the prediction confidence based on model probabilities.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask

- Created separate routes for Home (/), Prediction Page (/predict), and Result (/result).
- Used Flask's render_template() function to navigate between webpages.

Process User Input

- Accepted applicant information through HTML forms.
- Converted form values into a Pandas DataFrame.
- Applied the saved StandardScaler to numerical features.
- Passed the processed data to the trained Random Forest model.

Generate Prediction

- Predicted whether the application would be **Approved** or **Rejected**.
- Calculated prediction confidence using predict_proba().
- Displayed the final result on the Result page.

Milestone 3: Flask Application Development

In this milestone, the Flask web application was developed to integrate the trained Random Forest model with the user interface. The application accepts applicant information, processes the input, performs prediction using the trained model, and displays the approval result along with the confidence score.

Activity 3.1: Developing the Flask Application

Implemented Features

- Created the Flask application using Python.
- Loaded the trained Random Forest model and StandardScaler using Joblib.
- Defined routes for Home, Prediction, and Result pages.
- Connected the frontend forms with backend prediction logic.

app.py > ...

```python
from flask import Flask, render_template, request
import joblib
import pandas as pd

app = Flask(__name__)

# ================= LOAD MODEL =================

model = joblib.load("model/best_model.pkl")
scaler = joblib.load("model/scaler.pkl")

# Feature names (same order as training)
feature_names = [
    "CODE_GENDER",
    "FLAG_OWN_CAR",
    "FLAG_OWN_REALTY",
    "CNT_CHILDREN",
    "AMT_INCOME_TOTAL",
    "NAME_INCOME_TYPE",
    "NAME_EDUCATION_TYPE",
    "NAME_FAMILY_STATUS",
    "NAME_HOUSING_TYPE",
    "DAYS_BIRTH",
    "DAYS_EMPLOYED",
    "FLAG_MOBIL",
    "FLAG_WORK_PHONE",
    "FLAG_PHONE",
    "FLAG_EMAIL",
    "OCCUPATION_TYPE",
    "CNT_FAM_MEMBERS"
]

# Numerical columns that were scaled during training
numeric_cols = [
    "CNT_CHILDREN",
    "AMT_INCOME_TOTAL",
    "DAYS_BIRTH",
    "DAYS_EMPLOYED",
    "CNT_FAM_MEMBERS"
]
```

Input Processing and Prediction

The backend processes user input through the following steps:

- Reads applicant information submitted through the HTML form.
- Performs basic eligibility validation.
- Converts user input into a Pandas DataFrame.
- Applies StandardScaler to numerical features.
- Sends the processed data to the trained Random Forest model.
- Calculates prediction confidence using predict_proba().

```
# ================ HOME PAGE ================

@app.route("/")
def home():
    return render_template("home.html")

# ================ PREDICTION PAGE ================

@app.route("/predict")
def predict():
    return render_template("index.html")


# ================ RESULT PAGE ================

@app.route("/result", methods=["POST"])
def result():
```

Result Generation

The prediction module performs the following operations:

- Predicts whether the applicant is **Approved** or **Rejected**.
- Computes the confidence score.
- Displays the prediction result on the Result page.
- Handles invalid inputs using exception handling.

```
try:
    days_employed = float(request.form["f11"])

    if days_employed == 365243:
        days_employed = -1825

    values = [
        float(request.form["f1"]),
        float(request.form["f2"]),
        float(request.form["f3"]),
        float(request.form["f4"]),
        float(request.form["f5"]),
        float(request.form["f6"]),
        float(request.form["f7"]),
        float(request.form["f8"]),
        float(request.form["f9"]),
        float(request.form["f10"]),
        days_employed,
        float(request.form["f12"]),
        float(request.form["f13"]),
        float(request.form["f14"]),
        float(request.form["f15"]),
        float(request.form["f16"]),
        float(request.form["f17"])
    ]
except (ValueError, KeyError):
    return render_template(
        "result.html",
        prediction="Invalid Input",
        confidence=0
    )
```

Application Workflow

The complete application workflow is as follows:

1. User opens the Home page.
2. Applicant enters personal and financial details.
3. Flask validates the submitted information.
4. Numerical features are scaled using the saved StandardScaler.
5. The trained Random Forest model predicts the approval status.
6. The prediction result and confidence score are displayed to the user.

SKILL WALLET
A SmartBridge Product

Credit Card Eligibility Checker

Check whether your credit card application is likely to be approved using our Machine Learning prediction system.

Check Eligibility

Fast Prediction

Get an instant prediction within seconds using our trained model.

Secure

Your information is processed securely and is not permanently stored.

Accurate

Built using Machine Learning to assist in credit card approval prediction.

How It Works

1

Fill Details

2

Predict

3

Get Result

Milestone 4: Frontend Development

The frontend of the Credit Card Approval Prediction system was developed using HTML5 and CSS3 to provide a simple, responsive, and user-friendly interface. The application allows users to enter applicant information, submit the form, and view the prediction result generated by the machine learning model.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Developed the **home.html**, **index.html**, and **result.html** pages for navigation and prediction.
- Designed a user-friendly form to collect applicant information required for credit card approval prediction.
- Used semantic HTML elements to organize the interface and improve readability.

Design a Responsive Layout Using CSS:

- Created a dedicated **style.css** file for application styling.
- Designed responsive forms, buttons, and result sections for better usability.
- Ensured consistent layout and compatibility across different screen sizes.

Create User-Friendly Components:

- Added labeled input fields for applicant details.
- Included a prediction button to submit user information.
- Displayed prediction results clearly as **Approved** or **Rejected** along with confidence percentage.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission:

- Created an HTML form that collects applicant details and submits them to the Flask backend.
- Configured the form to send data to the **/result** route using the POST method.
- Validated required fields before processing the prediction.

Render Results Dynamically:

- Received prediction results from the machine learning model through Flask.
- Displayed the prediction status and confidence score on the result page.
- Allowed users to return to the prediction page and test multiple applications.

Maintain User Experience:

- Provided clear navigation between Home, Prediction, and Result pages.

- Displayed appropriate messages for invalid inputs

- Ensured smooth interaction between the frontend and backend throughout the prediction process.

Milestone 5: Deployment

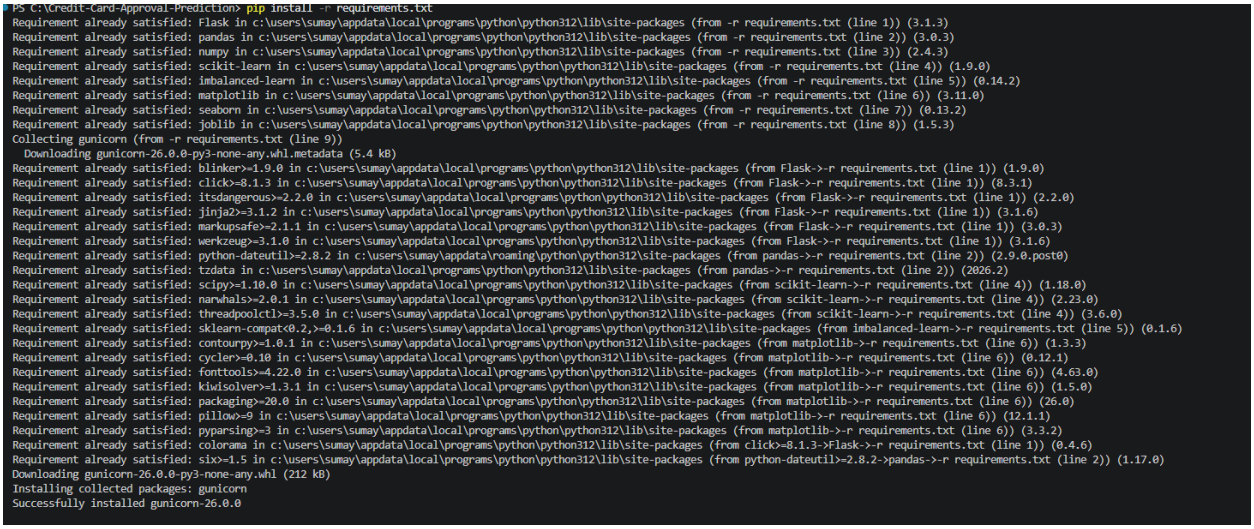
In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

Activity 5.1: Preparing the Application

Configure the Project Environment:

- Install all required Python libraries using the requirements.txt file.
- Ensure the trained model (**best_model.pkl**) and scaler (**scaler.pkl**) are available in the model folder.
- Verify that the dataset, templates, static files, and application files are organized correctly.

```
pip install -r requirements.txt
```



Start the Flask Application

Run the application using:

```
python app.py
```

The Flask development server starts successfully and hosts the application on the local machine.

```
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 376-154-741
```

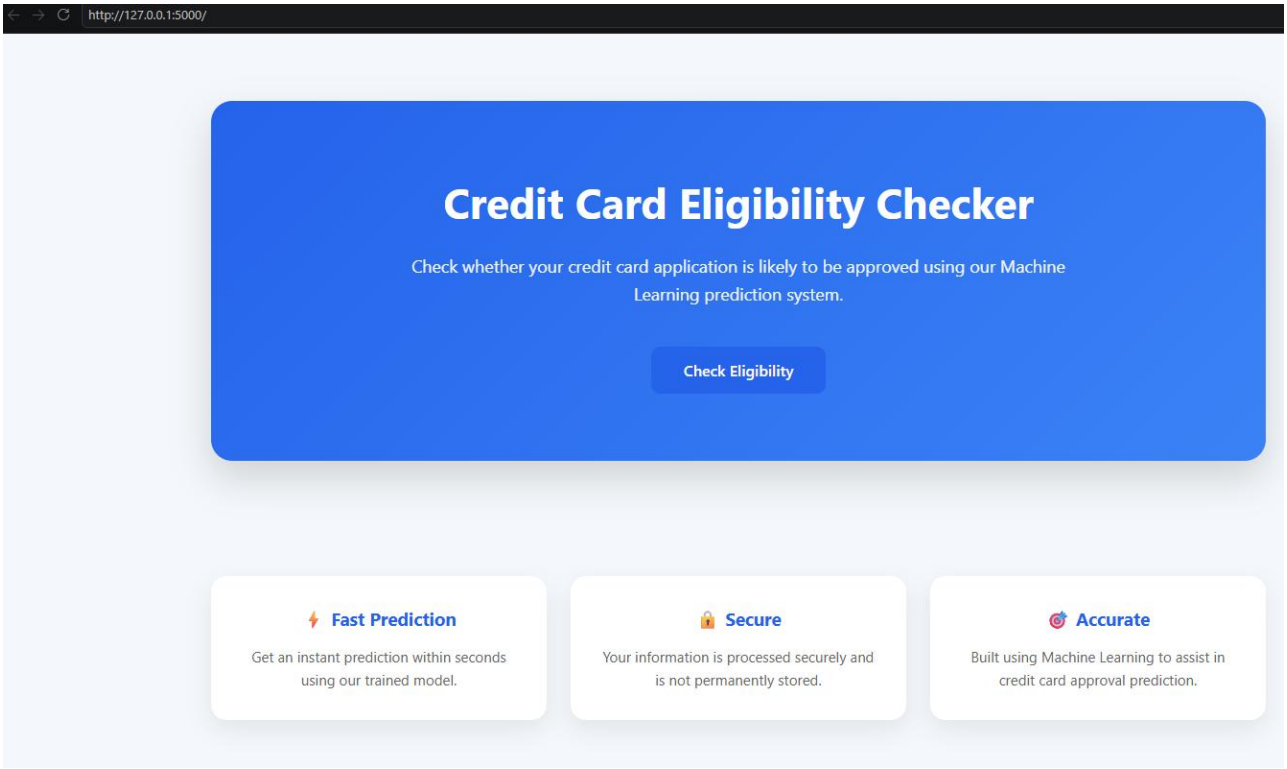
Activity 5.2: Local Testing and Verification

After starting the server, open the application in a web browser using:

`http://127.0.0.1:5000`

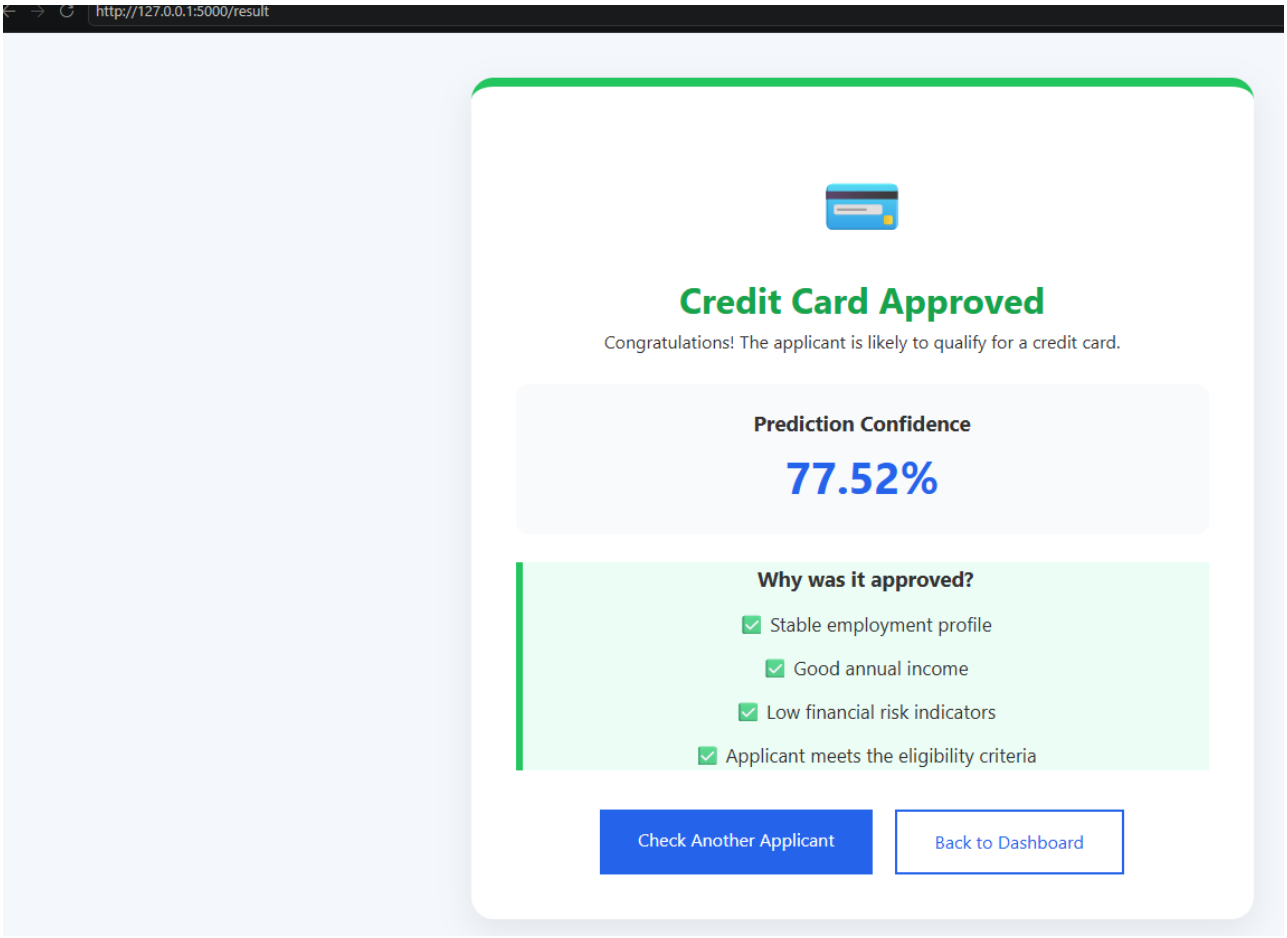
The following tests were performed:

- Verified that the Home page loads correctly.
- Verified navigation to the Prediction page.
- Entered applicant details and submitted the prediction form.
- Confirmed that the machine learning model returns either **Approved** or **Rejected**.
- Tested invalid and boundary inputs to ensure proper error handling.



Application Verification

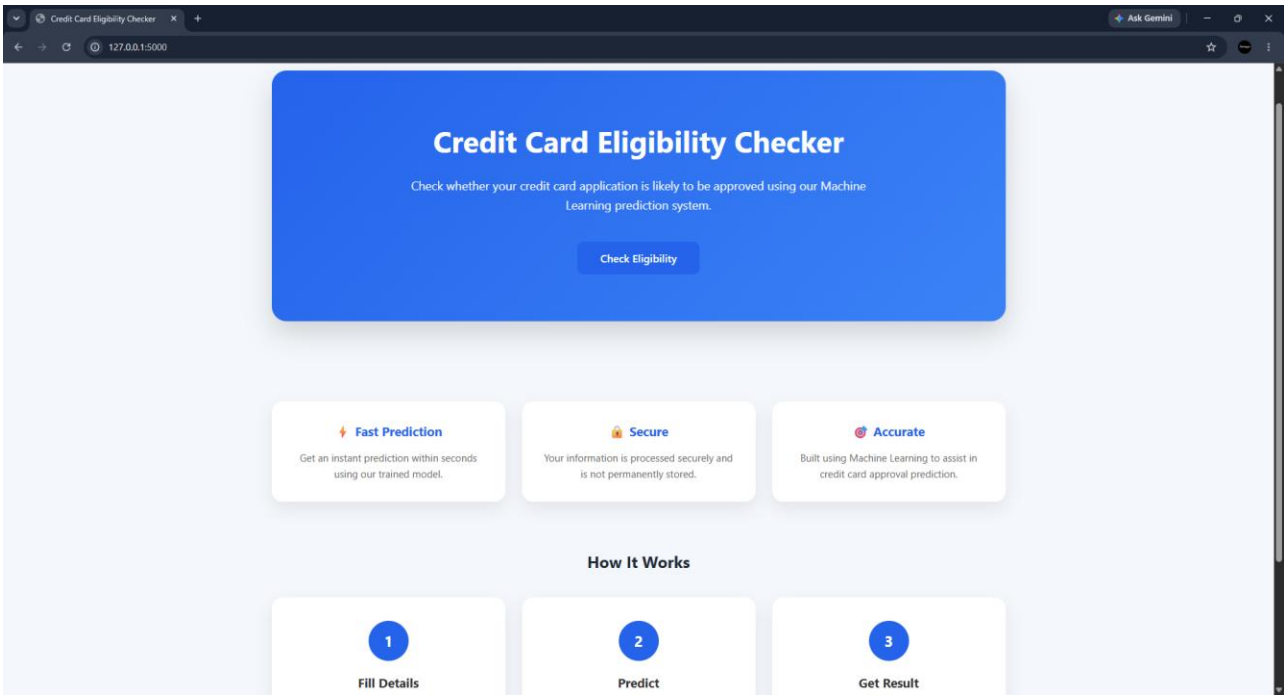
The application was successfully tested on the local system. The frontend communicates correctly with the Flask backend, and the trained Random Forest model produces prediction results with the calculated confidence score. All project modules function as expected.



Exploring the Website's Web Pages:

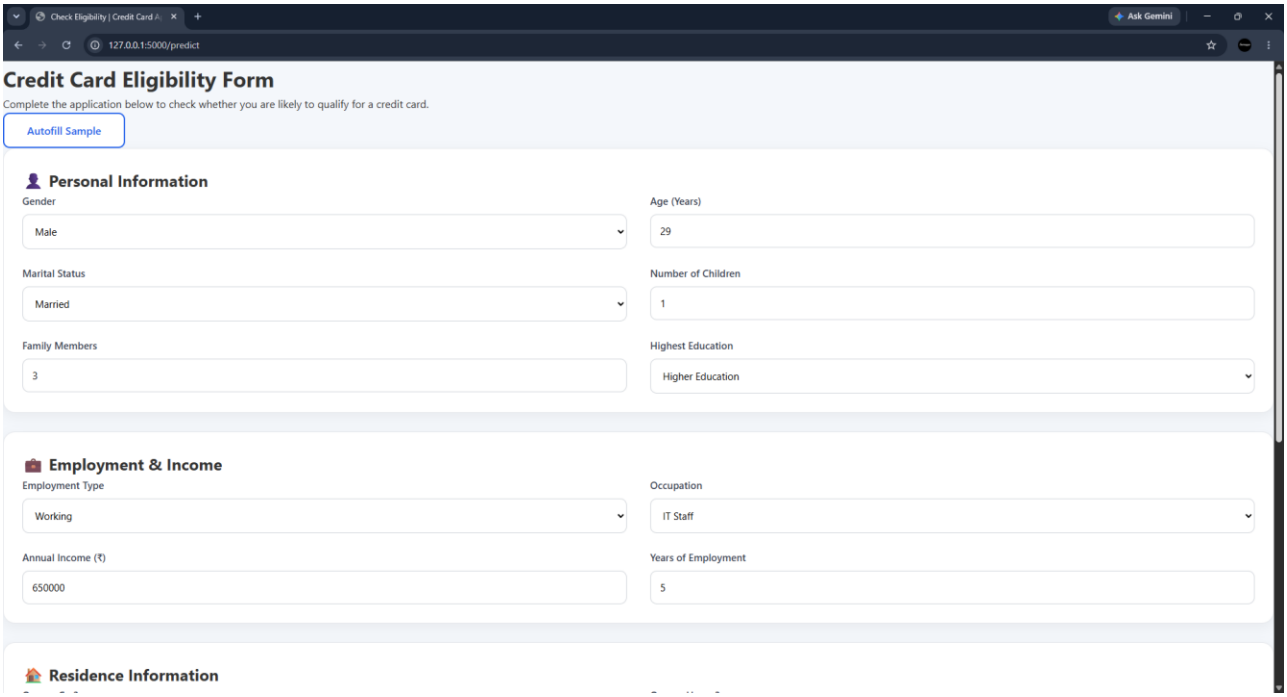
Home Page:

Description: The Home Page serves as the entry point of the Credit Card Approval Prediction System. It provides an overview of the application and allows users to navigate to the prediction page. The interface is designed with a clean and responsive layout, making it easy for users to understand the purpose of the system before starting the prediction process.



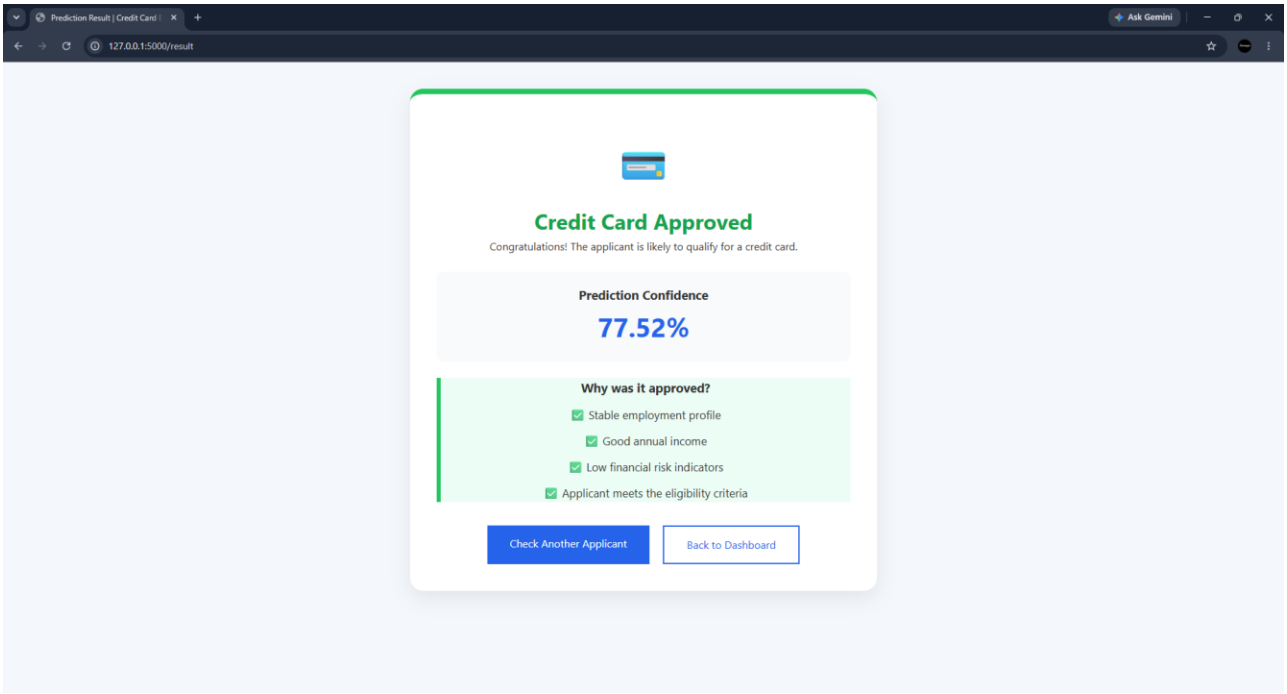
Input Section:

Description: The Prediction Page contains a form where users enter applicant details such as gender, income, employment status, education, family information, housing type, and other required attributes. After filling in all the necessary fields, users click the **Predict** button to submit the information to the trained machine learning model.



Results Section:

Description: After the prediction request is processed, the system displays the final result indicating whether the credit card application is **Approved** or **Rejected**. Along with the prediction, the application also displays the confidence score generated by the Random Forest model, helping users understand the reliability of the prediction.



Conclusion:

The **Credit Card Approval Prediction System** was successfully developed using **Python, Flask, HTML, CSS, JavaScript, and Machine Learning**. A trained **Random Forest Classifier** was integrated into the web application to predict whether a credit card application would be approved based on applicant information. The system provides an easy-to-use interface, accurate predictions, and confidence scores, demonstrating the practical application of machine learning in financial decision support systems.